Rule Engines

Intelligent Systems II

Mário Antunes

2026

Practice Sheet: Rule Engines

Objectives

- Understand the paradigm shift from Logic Programming (Prolog) to Rule Engines (GoRules).
- Learn how to deploy and interact with a Business Rules Management System (BRMS) via REST APIs.
- Model real-world legal regulations into visual decision tables.
- Integrate a stateless Rule Engine with a stateful IoT system using Python and MQTT.

Pre-requisites: Environment Setup

Before starting the exercises, you must spin up the GoRules BRMS environment provided in the `support` folder.

0. **Install docker:** Follow the instructions here, or try `docker_setup.sh` script in the support folder. **LICENSE_KEY:** Obtain it from https://portal.gorules.io (free account). Open your `docker-compose.yml` file and paste this key into the `LICENSE_KEY` environment variable.
1. **Start the Environment:** Navigate to the `support` folder and run the provided `Docker Compose` file: `bash docker compose up -d`
2. **Access GoRules:** Open your browser and go to `http://localhost:8080`.
3. **Generate API Token:** To communicate with the GoRules engine via Python, you need an API Key.
    - Inside the GoRules UI, create a new Project.

- Navigate to **Settings** → **Access Tokens**.
- Generate a new Personal Access Token. You will use this in your Python scripts in the `X-Access-Token` HTTP header.
- Take note of your **Project ID** (found in the URL of your project) as it is required for the API endpoints.

Exercise 1: Speeding Ticket System (Business Rules)

Rule engines are heavily used in the legal and compliance sectors. In this exercise, you will model the actual Portuguese traffic laws for speeding fines into a GoRules Decision Table.

Your Task: Create a Decision Table in GoRules that takes three inputs and returns the minimum fine amount and the severity of the infraction.

Inputs

- `vehicle_type` (String): "Light" (includes motorcycles) or "Heavy".
- `zone` (String): "Urban" (Inside localities) or "Rural" (Outside localities/Highways).
- `excess_speed` (Number): The speed in km/h *above* the legal limit.

Outputs

- `fine` (Number): The minimum monetary fine in Euros.
- `severity` (String): "Light", "Serious", or "Very Serious".

The Rules (Portuguese Highway Code)

1. Light Vehicles & Motorcycles (`vehicle_type: "Light"`)

Zone	Excess Speed (km/h)	Minimum Fine (€)	Severity
Urban	Up to 20	60	Light
Urban	21 to 40	120	Serious
Urban	41 to 60	300	Very Serious
Urban	> 60	500	Very Serious
Rural	Up to 30	60	Light
Rural	31 to 60	120	Serious
Rural	61 to 80	300	Very Serious

Zone	Excess Speed (km/h)	Minimum Fine (€)	Severity
Rural	> 80	500	Very Serious

2. Heavy / Other Vehicles (`vehicle_type: "Heavy"`)

Zone	Excess Speed (km/h)	Minimum Fine (€)	Severity
Urban	Up to 10	60	Light
Urban	11 to 20	120	Serious
Urban	21 to 40	300	Very Serious
Urban	> 40	500	Very Serious
Rural	Up to 20	60	Light
Rural	21 to 40	120	Serious
Rural	41 to 60	300	Very Serious
Rural	> 60	500	Very Serious

Test your rule in the GoRules UI simulator to ensure it correctly identifies a Heavy vehicle going 25 km/h over the limit in an Urban zone as a Very Serious infraction with a 300€ fine.

Exercise 2: Flappy Bird Agent with Rule Engine

In the previous practical sheet, you used Prolog to calculate when the Flappy Bird agent should jump. Now, you will replace Prolog with GoRules.

Your Task:

1. **The Rule:** In your GoRules project, create a new Decision Table.
   - **Inputs:** `bird_y` (Number) and `next_pipe_center_y` (Number).
   - **Output:** `action` (String).
   - **Logic:** Replicate the Prolog rule: If the `bird_y` is dangerously close to the bottom pipe (e.g., `bird_y > next_pipe_center_y`), the output should be `"jump"`. Otherwise, use a catch-all empty rule to output `"stay"`.
2. **The Python Script:** Open the provided `play_prolog.py` script.
   - Remove the `pyswip` library imports and logic.

- Use the `requests` library to send the `bird_y` and `next_pipe_center_y` values as a JSON payload to your GoRules API endpoint: `http://localhost:8080/a` `project-id}/evaluate/{your-document-path}`.
- Extract the `result` from the JSON response and trigger the websocket click command if the engine returns `"jump"`.

3. **Evaluate your Solution:** Run the game with the new agent (keep in mind that a REST API may not be speedy enough for the game).

Exercise 3: IoT Automation

Rule engines evaluate the *current* state of the world, meaning they are stateless. If you need historical context (like a sliding window of sensor data to avoid noisy readings), that logic must be handled outside the engine.

In this exercise, you will build a *robust* system where a Python script handles the state, and GoRules handles the decision.

For this exercise we need to connect to a custom network as seen in the next figure, credentials below:

Network Name (SSID)	Password
TheOffice	8006002030

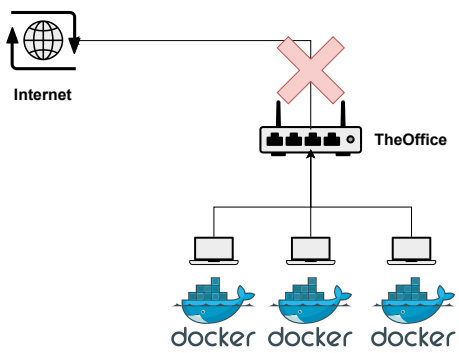

Figure 1: Custom network

Your Task:

1. **The Hardware/Microcontroller:** Wire up an RPi Pico with a DHT11 temperature/humidity sensor and an LED. Write MicroPython code to:
   - Read the humidity from the DHT11 every 1 second.

- Publish the raw reading as JSON to an MQTT topic (`sensors/pico`).
- Subscribe to an action topic (`actions/pico`) and turn the LED on/off based on the received command (`led: [0|1]`).

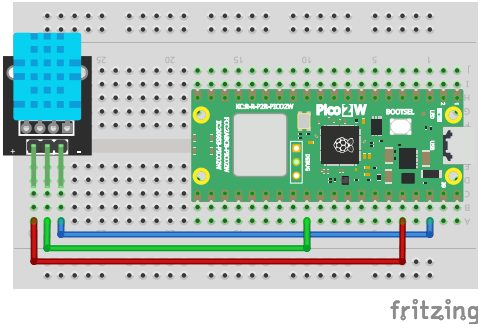

Figure 2: Circuit

2. **The Python Bridge (State Manager):** Write a Python script running on your computer that acts as a bridge:
   - Subscribe to the `sensors/pico` MQTT topic.
   - Maintain a sliding window (a list) of the last 5 humidity readings.
   - Calculate the `median_humidity`.
   - Send the `median_humidity` value to your GoRules API.
3. **The Rule Engine:** Create a Decision Table in GoRules that outputs `led: 1` if the median humidity is above a certain threshold (e.g., 60%), and `led: 0` otherwise.
4. **The Actuation:** The Python bridge receives the `led` value from GoRules, and publishes it to the `actions/pico` topic to trigger the physical hardware.

Appendix

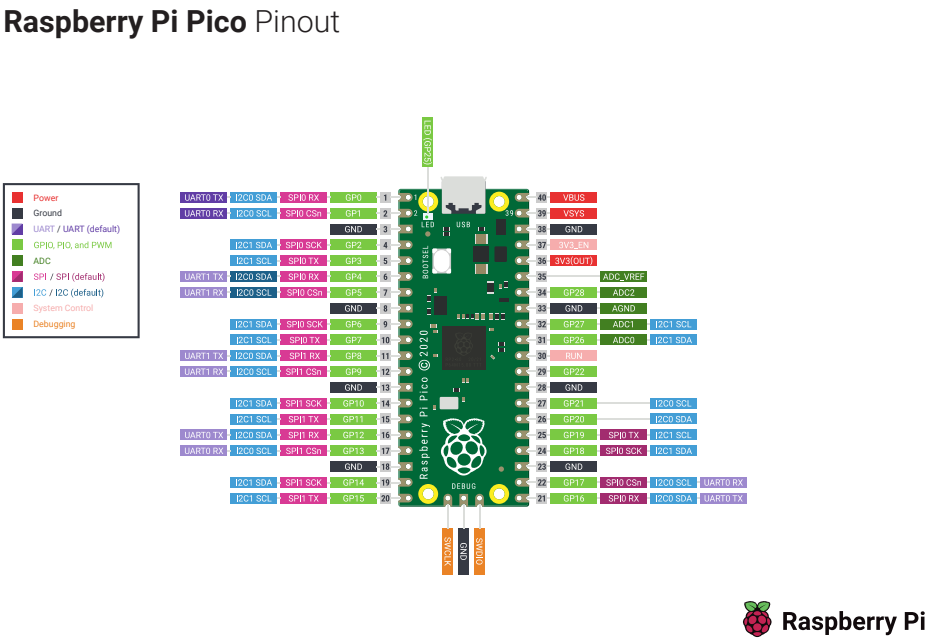

Figure 3: RPi Pico Pinout

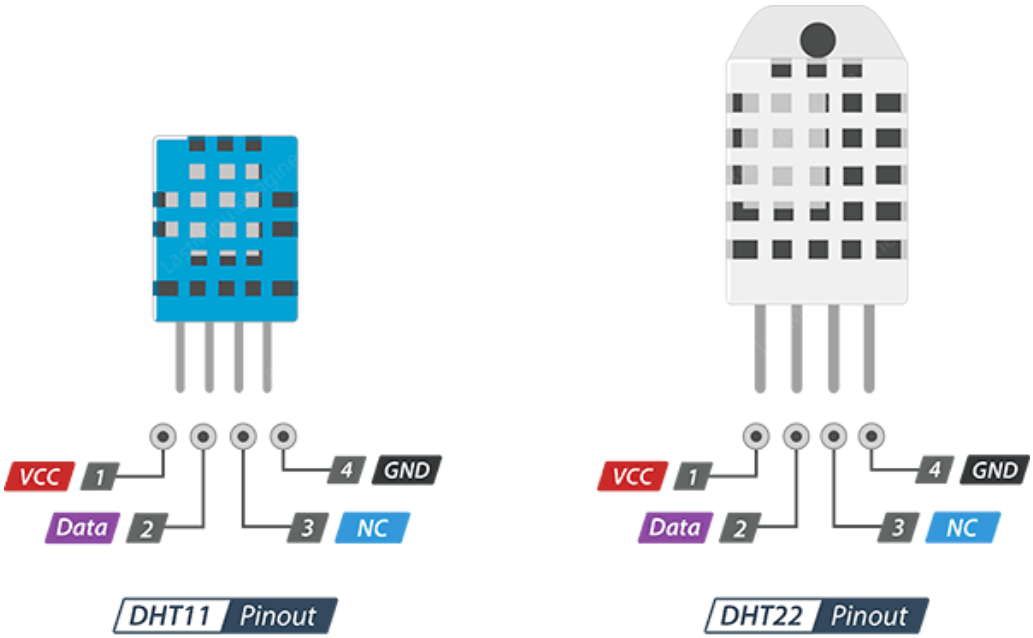

Figure 4: DHT Pinout